

AI-SA-IN-TH-000004

Module 8.1 · LLM Evaluation Fundamentals · AI Safety: Evaluation, Guardrails & Responsible Deployment

W-11 Start @ 11 am.

Perplexity and Generation Metrics

Automated signals for measuring fluency, similarity, and code correctness

9 min

Duration

B3 – Apply

Bloom's

L2 – Intermediate

Level

Theory

Domain

Learning Objectives

- 1 Define perplexity and compute it from a language model probability distribution
- 2 Explain the scaling law relationship between perplexity and model size
- 3 Apply ROUGE-L to a summarisation example and interpret precision, recall, and F1
- 4 Describe BERTScore's contextual embedding architecture and its advantage over ROUGE
- 5 Compute $\text{pass}@k$ using the unbiased estimator formula for code evaluation
- 6 Select the appropriate generation metric for a given task using the decision tree

Perplexity — Definition

Perplexity (PPL)

Perplexity measures how well a language model predicts a held-out text corpus. It is the exponentiated average negative log-likelihood per token: a lower perplexity means the model assigns higher probability to the test tokens — i.e., it is less 'surprised' by the text.

- Intuition: a model with $PPL = 10$ is as uncertain as if it were choosing uniformly among 10 equally probable next tokens at each step
- Directly measures language modelling quality — useful for comparing pre-trained models and tracking training progress
- Sensitive to tokeniser choice: PPL scores are not comparable across models with different tokenisers
- Reflects calibration of the probability distribution, not correctness or factuality of generated text

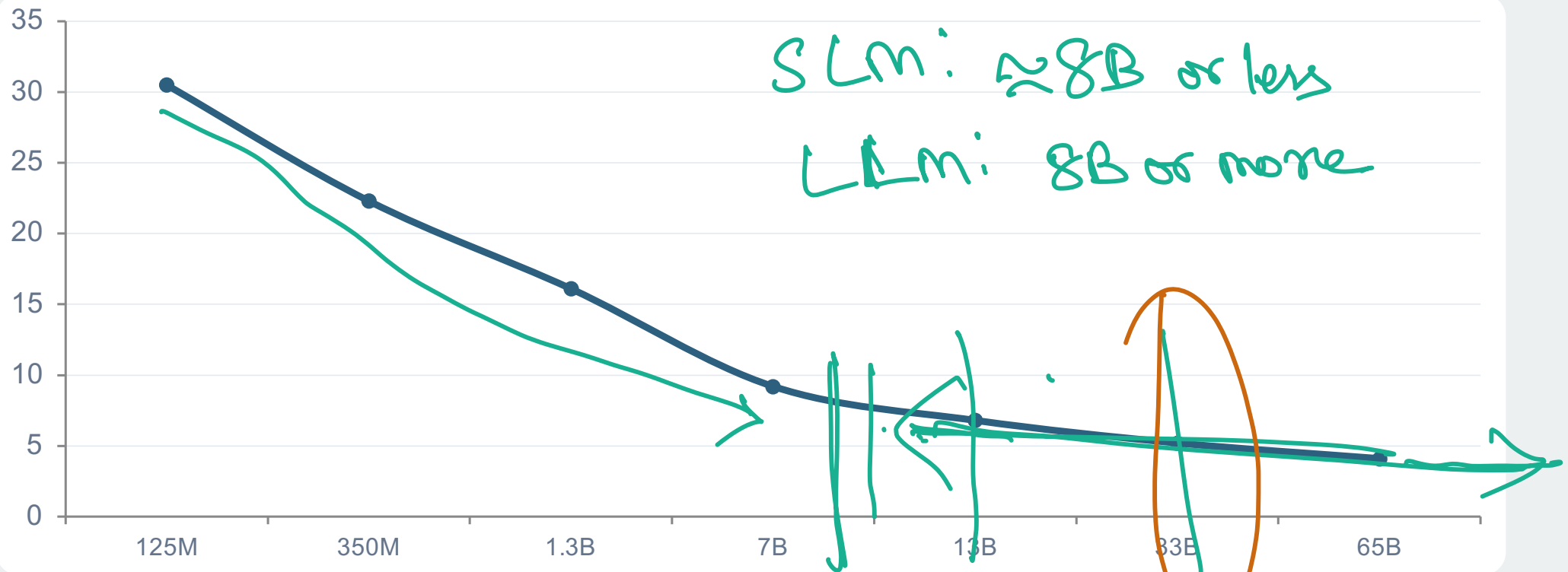
Perplexity — Formula and Worked Calculation

$$\text{PPL}(W) = \exp(-1/N \cdot \sum \log P(w_i | w_1 \dots w_{i-1}))$$

1. Given test sequence: $W = [\text{"The"}, \text{"cat"}, \text{"sat"}]$ with $N = 3$ tokens
2. Model assigns log-probabilities: $\log P(\text{The}) = -1.2$, $\log P(\text{cat}|\text{The}) = -2.1$, $\log P(\text{sat}|\text{The cat}) = -1.8$
3. Average NLL = $-(1/3) \times (-1.2 + -2.1 + -1.8) = -(1/3) \times (-5.1) = 1.70$
4. Perplexity = $\exp(1.70) \approx 5.47$ — the model is as uncertain as a uniform distribution over ~5.5 tokens per step
5. For a 7B LLM evaluated on WikiText-103: GPT-2 XL ≈ 18.3 , LLaMA-2-7B ≈ 5.47 , LLaMA-3-8B ≈ 3.98

Note: PPL is not comparable across different tokenisers — always normalise by bytes or characters for fair comparison

Perplexity vs Model Size — Chinchilla Scaling Law



Source: Chinchilla (Hoffmann et al., 2022) and LLaMA (Touvron et al., 2023). Log-linear relationship holds across model families. Beyond ~65B, returns diminish rapidly.

What Perplexity Doesn't Capture

Perplexity is a necessary but far from sufficient evaluation signal:

- Factual accuracy: a fluent, confident, but entirely wrong statement scores as well as a correct one
- Task performance: a model can have low PPL on general text but fail completely at code, math, or instruction-following
- Safety: harmful, toxic, or biased content can be assigned high probability by a well-trained language model
- Instruction alignment: PPL measures fit to the training distribution, not ability to follow user instructions
- Reference independence: PPL requires a held-out corpus — it cannot evaluate a single model response against a desired output
- Summary: use PPL for pre-training tracking and model-family comparisons; never use PPL alone to assess deployment readiness

ROUGE-L — Definition

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation — Longest Common Subsequence)

ROUGE-L measures the length of the longest common subsequence (LCS) between a candidate summary and a reference summary. Unlike ROUGE-N, it does not require n-grams to be contiguous, capturing sentence-level word order similarity.

- Recall (R): $\frac{\text{LCS length}}{\text{reference length}}$ — did the candidate cover the key content? .63 1 1
- Precision (P): $\frac{\text{LCS length}}{\text{candidate length}}$ — was the candidate concise without padding? < < 1 X
- F1 score: harmonic mean of R and P — standard combined metric for automatic summarisation evaluation
- Widely used for summarisation (CNN/DailyMail), machine translation, and question-answering benchmarks
- Does not require exact n-gram match — captures long-range word order alignment across sentence structure

$$\frac{1}{100} \quad 441$$

ROUGE-L — Worked Summarisation Example

Evaluate a model-generated summary against the reference summary

Reference Summary

"The cat sat on the warm mat by the window." = 9

Candidate Summary

"A cat sat near the window on a mat."

LCS: [cat, sat, the, window, mat] → length = 5

Recall $R = 5 / 9 = 0.556$ (reference has 9 tokens)
Precision $P = 5 / 9 = 0.556$ (candidate has 9 tokens)
ROUGE-L F1 = $2 \times 0.556 \times 0.556 / (0.556 + 0.556) = 0.556$

Interpretation: F1 = 0.556 indicates moderate coverage of key content. A score > 0.4 is generally considered acceptable for news summarisation tasks.

ROUGE-L — Strengths vs Limitations

Strengths

- Fast to compute — no model inference required
- Interpretable: directly measures token-level overlap
- Does not require contiguous n-grams — handles paraphrase partially
- Standard metric across summarisation benchmarks (CNN/DM, XSum) — enables reproducible comparison
- Robust for extractive summaries where surface-level overlap is meaningful

Limitations

- Paraphrase-blind: 'automobile' and 'car' score zero overlap despite synonymy
- Rewards verbosity: longer candidates with more overlapping words inflate recall
- Ignores semantics: factually wrong summaries can score highly with correct vocabulary
- Poor for abstractive summaries that paraphrase significantly
- Human correlation: ROUGE-L correlates with human judgment only ~0.5–0.6 on standard benchmarks

BERTScore — Definition

BERTScore (Zhang et al., 2019)

BERTScore evaluates text generation quality by computing token-level cosine similarity between contextual BERT embeddings of the candidate and reference texts. Unlike ROUGE, it captures semantic similarity — synonyms and paraphrases receive high scores even without surface-level token overlap.

- Uses pre-trained BERT (or RoBERTa, DeBERTa) to embed each token in context
- Precision: for each candidate token, find the most similar reference token by cosine similarity
- Recall: for each reference token, find the most similar candidate token
- F1: harmonic mean of BERTScore precision and recall
- Correlates with human judgment significantly better than ROUGE-L (0.7–0.8 vs 0.5–0.6)

BERTScore — Contextual Embedding Architecture

Candidate: The

cat

sat

near

the

window

BERT Encoder → Contextual Embeddings

Reference: The

cat

sat

on

the

mat

Cosine Similarity Matrix → Greedy Token Matching → Precision / Recall / F1

0.91

BERTScore Precision

0.88

BERTScore Recall

0.89

BERTScore F1

BERTScore vs ROUGE-L vs BLEU — Comparison

Property	BLEU	ROUGE-L	BERTScore
Matching type	N-gram exact match	LCS token overlap	Contextual cosine sim
Handles paraphrase	✗ No	✗ Partial	✓ Yes
Requires reference	✓ Yes	✓ Yes	✓ Yes
Computation cost	Negligible	Negligible	Moderate (BERT inference)
Human correlation	~0.4–0.5	~0.5–0.6	~0.7–0.8
Best use case	MT (historical)	Summarisation	All generation tasks
Current status (2026)	Deprecated for MT	Still standard	Preferred for semantics

BLEU — Historical Context and Why It's Being Retired

Case: BLEU (Bilingual Evaluation Understudy) — Papineni et al., 2002

- Proposed in 2002 as the first automatic MT evaluation metric — a landmark achievement enabling rapid MT research iteration
- BLEU measures geometric mean of 1–4 gram precision with a brevity penalty to discourage too-short translations
- Dominated MT evaluation for 20 years: WMT shared tasks, research papers, and industrial MT pipelines all reported BLEU
- Modern findings (2022–2026): BLEU correlates poorly with human judgment on neural MT outputs (Spearman $r \sim 0.4$)
- Neural MT systems game BLEU: they produce fluent, common-phrase-heavy outputs that score well but miss meaning

Key Lesson

WMT 2022 officially retired BLEU as the primary metric, replacing it with COMET and chrF. For new projects: use BERTScore for general text, COMET for translation, and task-specific metrics for other modalities. Report BLEU only for historical comparison.

pass@k — Code Generation Evaluation

pass@k

pass@k measures the probability that at least one of k independently sampled code completions from a model passes all unit tests for a given programming problem. It is the standard metric for code generation evaluation (HumanEval, MBPP, SWE-bench).

- pass@1: the model's single-shot success rate — used for greedy or temperature=0 generation
- pass@10 and pass@100: measure of problem-solving breadth — how reliably can the model solve problems given multiple attempts
- A direct, objective measure: unit tests are deterministic and unambiguous
- Cannot capture partial credit, code style, efficiency, or security considerations
- Sensitive to problem difficulty: pass@1 = 90% on simple problems; drops to ~15% on complex algorithmic tasks

pass@k — HumanEval Problem Walkthrough

Python / HumanEval

```
# HumanEval Problem HE-001: has_close_elements
# Task: Return True if any two numbers in list are closer than threshold

def has_close_elements(numbers: List[float], threshold: float) -> bool:
    for i in range(len(numbers)):
        for j in range(i+1, len(numbers)):
            if abs(numbers[i] - numbers[j]) < threshold:
                return True
    return False

# Unit tests (hidden from model during generation):
# assert has_close_elements([1.0, 2.0, 3.9, 4.0], 0.05) == True
# assert has_close_elements([1.0, 2.0, 3.0], 0.5) == False
# pass@1 = 1.0 for GPT-4 on this example (trivial)
# pass@1 = 0.0 on harder problem HE-148 (even GPT-4 fails)
```

HumanEval: 164 hand-crafted Python problems with 7.7 tests/problem avg. GPT-4 pass@1 $\approx$ 90.2% (2024). Best open model: DeepSeek-Coder-V2 $\approx$ 90.7%.

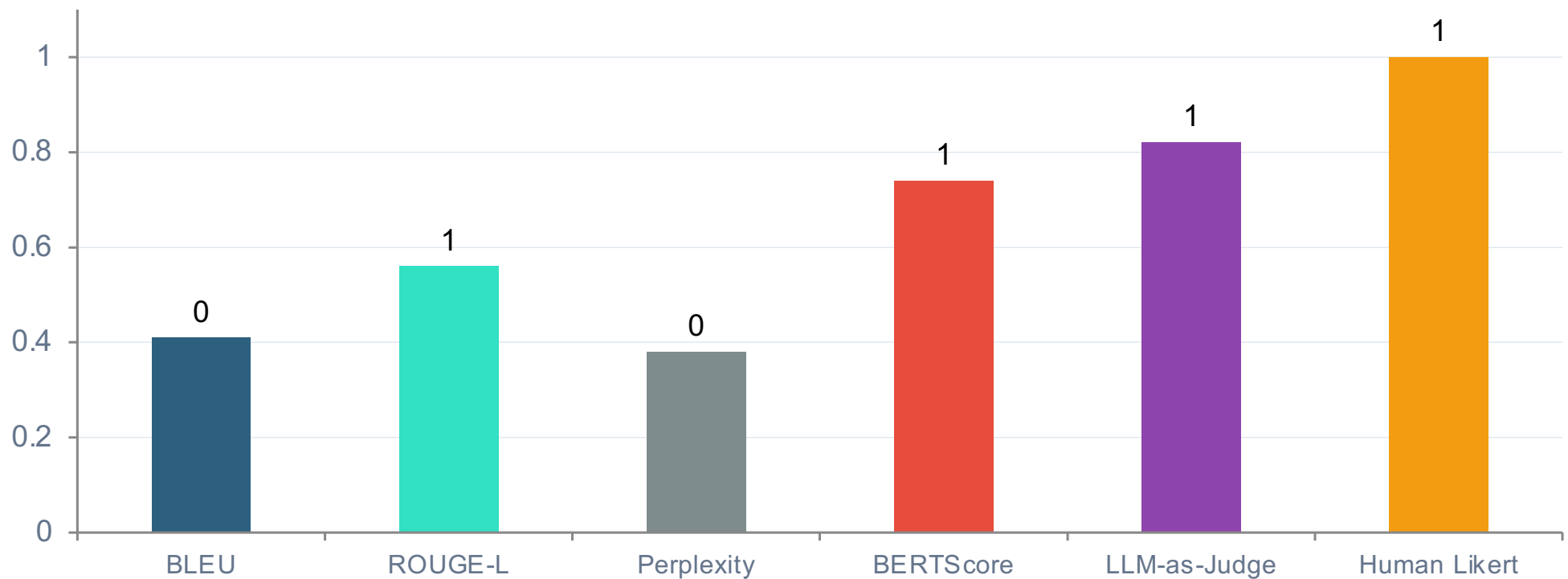
pass@k — Unbiased Estimator Formula

$$\text{pass@k} = 1 - C(n-c, k) / C(n, k)$$

1. Generate n samples per problem; c = number of samples that pass all unit tests
2. Compute the probability that at least one of k randomly chosen samples passes
3. Worked example: $n=10$ samples, $c=4$ pass, $k=1 \rightarrow \text{pass@1} = 1 - C(6,1)/C(10,1) = 1 - 6/10 = 0.40$
4. Same example, $k=5 \rightarrow \text{pass@5} = 1 - C(6,5)/C(10,5) = 1 - 6/252 = 0.976$
5. This estimator is unbiased: it avoids the naïve $(c/n)^k$ estimator which is biased for small n

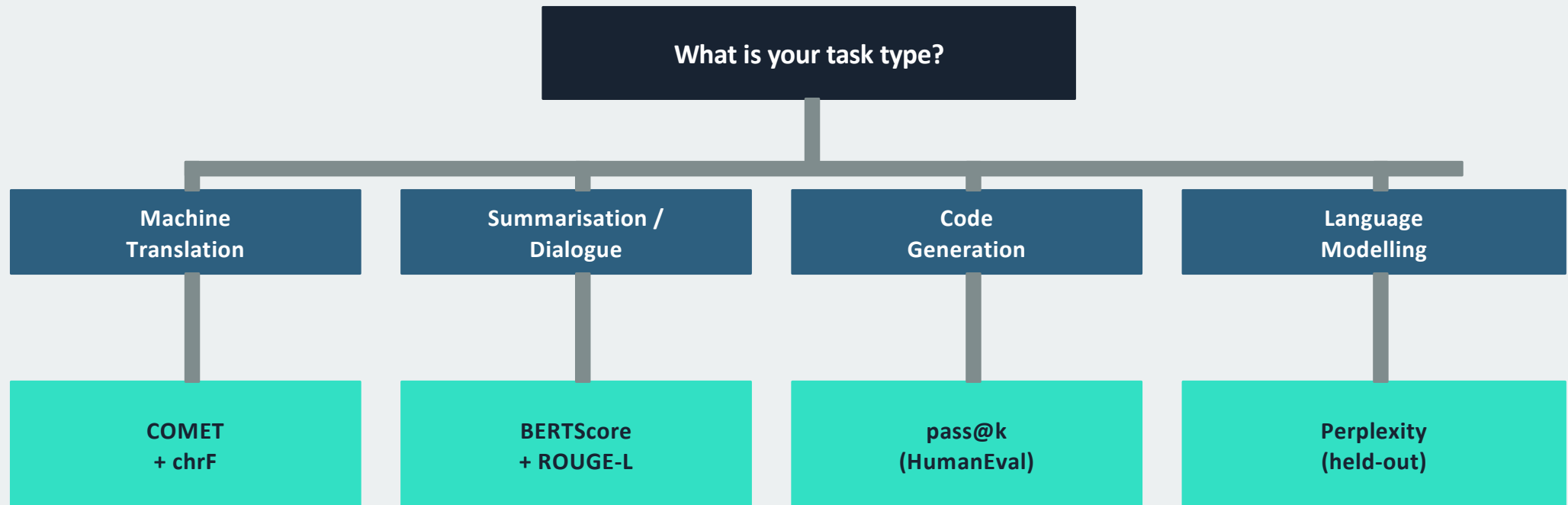
Note: Always report n (sample count) alongside pass@k . Results with $n=10$ and $n=200$ are not directly comparable.

Metric-to-Human-Preference Correlation (Spearman r)



Source: Meta-evaluation studies (Freitag et al. 2022, Fu et al. 2023). LLM-as-judge closes the gap with human evaluation at a fraction of the cost. Use BERTScore as your baseline automated metric; supplement with LLM-as-judge for nuanced quality.

Metric Selection Decision Tree



For all tasks: supplement automated metrics with LLM-as-judge or human evaluation for final release decisions.

Key Takeaways

AI-SA-IN-TH-000004

- 1 Perplexity measures prediction confidence — lower is better — but captures nothing about factuality, safety, or instruction-following
- 2 ROUGE-L measures token-level LCS overlap: fast and interpretable, but paraphrase-blind and poorly correlated with human judgment (~ 0.56)
- 3 BERTScore uses contextual BERT embeddings for semantic similarity — the current best automated metric (human correlation ~ 0.74)
- 4 BLEU is deprecated for modern NLP — use COMET or chrF for translation, BERTScore for generation
- 5 pass@k is the gold standard for code evaluation: objective, deterministic, and directly tied to functional correctness